



Vetores e cadeias de texto (Strings)



Vetor / Array

Um vetor ou array é uma estrutura de memória que permite armazenar um conjunto agrupado de valores do mesmo tipo.

- Para defini-lo, você especifica o tipo de dados dos valores que ele conterá e sua capacidade, que será o número de valores que ele será capaz de armazenar.
- Cada elemento do array é acessível por meio do nome do array e da posição que ocupa dentro dele.
- A primeira posição de um array será sempre a posição 0.



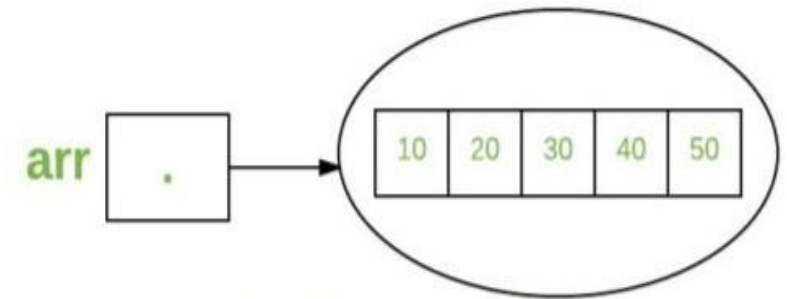
40	55	63	17	22	68	89	97	89
0	1	2	3	4	5	6	7	8

<- Array Indices

Array Length = 9

First Index = 0

Last Index = 8



One-Dimensional Array

Ejemplo de array (Fuente: [geeksforgeeks.org](https://www.geeksforgeeks.org/))

Tokio.



Declarar um vetor

- Vetor de inteiros. Sem tamanho. Sem instanciar

```
int notas[];
```

- Vetor de inteiros. Tamanho 2, inicializado (padrão = 0)

```
int[] notas = new int [2];
```

- Vetor de strings. Tamanho 2, inicializado com valores

```
String[] palavras = new String[] {"um", "dois"};
```



Também é comum declarar um vetor e inicializá-lo usando um loop for. Você pode, por exemplo, pedir ao usuário para inserir cada valor:

```
Scanner teclado = new Scanner(System.in);  
int[] notas = new int[3];  
for (int i = 0; i < notas.length; i++) {  
    System.out.print("Introduza a nota " + i + ": ");  
    notas[i] = Integer.parseInt(teclado.nextLine());  
}
```



Como acessar os valores de um vetor

- É possível acessar todos os valores de um vetor usando um loop for para acessar posição a posição (maior controle)

```
for (int i = 0; i < notas.length(); i++) {  
    System.out.println(notas[i]);  
}
```
- Cada elemento de um vetor pode ser acessado usando um loop for-each (do início ao fim)

```
for (int nota : notas) {  
    System.out.println(nota);  
}
```



- Também é possível acessar qualquer posição diretamente
`int[] notas = new int[2];`
`notas[0] = 10;`
`notas[1] = 5;`
`System.out.println("Nota primeiro exame: " + notas[0]);`
`System.out.println("Nota segundo exame: " + notas[1]);`



Operações com vetores

Em Java existe a classe Arrays para realizar operações com vetores:

- `int binarySearch(vector[], valor)`: Encontra o valor especificado no vetor e retorna sua posição. Se não encontrá-lo retornará o valor -1
- `vector[] copyOf(vector[], novoTamanho)`: corta ou alonga o vetor que é passado como parâmetro para ajustar ao novo comprimento. Retorna o novo vetor modificado.
- `fill(vector[], valor)`: Preenche o vetor que é passado como um parâmetro com o valor especificado em todas as suas posições
- `sort(vector[])`:: Organiza todos os valores do vetor
- `String toString(vector[])`: Retorna o conteúdo do vetor representado por uma string



Vetores bidimensionais

- Vetores bidimensionais também são conhecidos como matrizes
- Eles funcionam da mesma forma que vetores unidimensionais, mas, neste caso, existem duas posições para indicar: linha e coluna

```
int[][] valores = new int[2][5]
```

- Eles também podem ser considerados como "vetores de vetores", uma vez que cada posição do vetor contém outro vetor



Exercícios com vetores

1. Escreva um programa que solicite 5 notas do usuário e calcule a nota média
2. Escreva um programa que solicite 5 notas do usuário e calcule a nota média, mas apenas para aquelas notas que são superiores a 5
3. Escreva um programa que, a partir de um vetor de palavras, busque uma palavra inserida pelo usuário e indique se ela está lá ou não no vetor.
4. Escreva um programa que permita ao usuário inserir quantos números desejar. Finalmente, você terá que exibir o número mais alto
5. Escreva um programa que gere uma série de 10 números aleatórios ordenados



Strings

- Uma cadeia de texto é uma sequência de caracteres (qualquer)
- Em Java, eles são definidos como o tipo String e seu tamanho é variável (e não é necessário especificá-lo)
- Em Java é permitido atribuir um valor a uma String como se fosse um tipo primitivo, mas realmente é um objeto.
- A string de texto é um objeto imutável, de modo que todas as operações que invocamos nele nunca o modificam, mas sim eles retornam seu valor modificado (que terá que ser armazenado sobre ele mesmo ou outra string)



Declarar uma cadeia de texto

- String de texto com valor nulo (é a forma de especificar que não tem valor)
`String nome = null;`
- String de texto vazia (é um valor válido)
`String name = "";`
- String de texto com valor
`String nome = "João Pedro";`
- Sequência de texto composta por dígitos (ainda é uma sequência de texto)
`String numero = "123";`



Quando utilizar String

Usaremos uma string quando quisermos armazenar texto.

Às vezes, mesmo quando o valor a ser armazenado consiste exclusivamente em dígitos, ele deve ser armazenado como string, uma vez que não são realmente valores numéricos, já que não suportam operação matemática.

Estes são alguns exemplos:

- NIF
- Número segurança social
- Conta bancária
- Telefone



Se fosse necessário realizar alguma operação com seus dígitos (calcular dígitos de verificação, por exemplo) você sempre pode recorrer a análise de valor para realizar esses cálculos.

Em suma, deve-se levar em consideração que todas as entradas do usuário (tanto do console quanto através dos formulários) sempre vêm na forma de texto e somos nós que temos que analisá-lo para o tipo correto (após as verificações)



Funções com Strings

A classe String contém uma série de métodos que podem ser usados no valor da string (e sempre retorna o resultado depois de aplicar este método):

- `boolean equals(outraString)`: Compara o valor da string com outro e retorna `true` se eles forem iguais e `false` em qualquer outro caso
- `boolean contains(subString)`: verifica se a substring que é interrompida como um parâmetro está contida na string
- `boolean endsWith(subString)`: verifica se a string termina com a substring que é interrompida como parâmetro



- `int indexOf(subString)`: Retorna a posição da substring que é passada como um parâmetro
- `boolean isEmpty()`: verifica se a string está vazia (comprimento = 0)
- `int length()`: Retorna o comprimento da string (número de caracteres)
- `String replace(subString, mudança)`: substitui a substring pela mudança que também é passado como um parâmetro, e o resultado é retornado
- `String[] split (regex)`: Retorna uma matriz com os fragmentos resultantes da divisão da string de acordo com a expressão regular o que é passado como parâmetro



- `char[] toCharArray()`: Retorna a string como um array de char
- `String toLowerCase()`: Retorna a string com todos os seus caracteres em minúsculas
- `String toUpperCase()`: Retorna a string com todos os seus caracteres em maiúsculas
- `String trim()`: Retorna a mesma string, eliminando todos os espaços no início e no final dela.
- `String valueOf(value)`: Devolve uma representação em String do valor que é passado como parâmetro (admite qualquer tipo primitivo)



Exercício de Strings

1. Escreva um programa por meio do qual um usuário possa jogar o jogo da forca. O computador terá que pensar em uma palavra e o usuário terá que adivinhá-lo propondo, por sua vez, caracteres para completá-lo. O usuário terá um número limite de tentativas e deve-se levar em consideração que se o caractere aparecer mais de uma vez na palavra, todos os casos devem ser mostrados.



Dúvidas?

Tokio.